



Anexo a la Memoria: Corrección de Errores

Refactorización de la Capa Staging (Fase C)

Contexto del documento: Este anexo documenta la corrección arquitectónica solicitada durante la revisión del proyecto. Sustituye el planteamiento original de la *Parte C: Capa Staging* detallada en la primera versión de la memoria.

1. EL PLANTEAMIENTO ORIGINAL (ERRÓNEO)

En la primera entrega del proyecto, la capa de Staging se implementó utilizando **VISTAS (VIEWS)**. El razonamiento inicial fue evitar la duplicidad de datos en el disco y ahorrar espacio de almacenamiento en Snowflake, creando las vistas como un "filtro" lógico que limpiaba los datos (por ejemplo, extraer solo los números del campo O_CLERK) leyendo directamente desde la capa RAW.

```
/* ENFOQUE ANTERIOR (Vistas lógicas) */ CREATE OR REPLACE VIEW STG_ORDERS AS SELECT O_ORDERKEY,  
O_CUSTKEY, O_ORDERSTATUS, O_TOTALPRICE, CAST(REPLACE(O_CLERK, 'Clerk#', '') AS NUMBER) AS  
O_CLERK_ID FROM PRACTICA_FINAL_DAW.RAW.RAW_ORDERS;
```

2. EL PROBLEMA DE RENDIMIENTO

Durante la revisión por parte del supervisor, se detectó que este enfoque **rompe el propósito principal de un proceso ETL real**.

Al utilizar vistas para aplicar transformaciones (como REPLACE, CAST, o conversiones de fechas), los datos no se limpian de forma permanente. En su lugar, **Snowflake se ve obligado a recalcular estas transformaciones cada vez que la capa WAREHOUSE hace una consulta**. En un entorno de producción con millones de registros, procesar estas reglas de limpieza "al vuelo" de forma repetida genera un cuello de botella computacional enorme y un consumo excesivo de créditos de procesamiento.

3. LA SOLUCIÓN: MATERIALIZACIÓN ETL

Para solucionar este error, se ha refactorizado completamente el script `Stagin.sql`. Se ha implementado una **Materialización ETL real** sustituyendo la creación de vistas lógicas por la creación de **tablas físicas**.

Para ello, se utilizó la instrucción **CTAS (Create Table As Select)**. De esta manera, los datos sucios de la capa RAW pasan por el "filtro" de limpieza **una sola vez**, y el resultado ya limpio y estructurado se guarda físicamente en disco dentro de las tablas de STAGING.

```
/* NUEVO ENFOQUE (Tablas Físicas - Materialización) */ CREATE OR REPLACE TABLE STG_ORDERS AS
SELECT O_ORDERKEY, O_CUSTKEY, O_ORDERSTATUS, O_TOTALPRICE, CAST(REPLACE(O_CLERK, 'Clerk#', '') AS
NUMBER) AS O_CLERK_ID FROM PRACTICA_FINAL_DAW.RAW.RAW_ORDERS;
```

Registro del Commit en GitHub

Este cambio ha sido reflejado en el repositorio oficial mediante el commit "**Update Stagin.sql**", con la siguiente descripción técnica:

"Se hace el staging en las propias tablas (Materialización ETL) en vez de en las vistas. Con esto ejecutado, los datos sucios de la capa RAW pasarán por el 'filtro' y se quedarán guardados físicamente y limpios dentro de las tablas de STAGING."

Impacto en la Arquitectura

Gracias a este ajuste, la tabla de hechos (`FACT_VENTAS`) de la capa Warehouse ahora lee datos que ya están consolidados y limpios. Esto ha reducido drásticamente los tiempos de lectura y procesamiento demostrados en la *Fase I (Análisis de Rendimiento)*, consolidando una arquitectura mucho más robusta y alineada con las mejores prácticas del sector del Data Engineering.